

Synthetic Space-Debris Risk Monitor

Faculty-facing explanation, technology rationale, working methodology, and demonstration guide

PROJECT TYPE	IMPLEMENTATION	DATA MODE
Educational MVP	Python + Streamlit	Synthetic orbit data

Prepared as a transparent prototype for demonstrating how a space-debris monitoring workflow can turn orbital object data into a prioritized close-approach alert.

Scope statement

OrbitGuard is a research demonstrator, not a flight-safety system. It is designed to explain the problem, show the calculation flow, and make assumptions visible before live data and mission-grade validation are introduced.

1. Faculty-ready explanation

Use the following explanation during the project introduction:

OrbitGuard is an early-stage decision-support prototype for space-debris monitoring. It uses a controlled synthetic dataset to demonstrate the workflow without claiming access to live spacecraft telemetry. The system represents satellites and debris fragments, estimates their closest approach using a simplified relative-motion model, ranks encounters using a transparent risk score, and visualizes the highest-priority hazard zone. The next research step would be integrating validated TLE or ephemeris data, uncertainty covariance, and production-grade collision-probability algorithms.

Problem statement

Spacecraft in low Earth orbit share crowded orbital regions with inactive satellites, rocket bodies, and fragmentation debris. Operators need to know which nearby object deserves attention, how soon a close approach may occur, and why one event is ranked above another. OrbitGuard demonstrates this prioritization process in a small, inspectable dashboard.

Objectives

- Create a synthetic catalog containing satellites and debris objects.
- Estimate time to closest approach and minimum relative distance.
- Calculate a simple, explainable risk score for triage.
- Present alerts, an orbital map, and a searchable tracking table.
- Document limitations clearly so the prototype is not mistaken for a live safety tool.

Expected outcome

A faculty member can open the dashboard, see the object catalog, understand the risk ranking, and trace the result back to a documented mathematical model.

2. Technologies and why they were chosen

Technology	Role	Why it was chosen
Python	Core language and calculations	Readable scientific syntax, strong mathematics ecosystem, and widely used in aerospace data analysis.
Streamlit	Dashboard interface	Turns a Python script into an interactive web UI without requiring JavaScript or a separate frontend framework.
Pandas	Tracking table	Provides clean tabular representation for object metadata and calculated encounter results.
Matplotlib	Orbital map	Creates a reproducible 2D visualization directly from Python and supports research-style plotting.
Synthetic data	Controlled input	Allows a safe demonstration while avoiding unsupported claims about live telemetry or operational data.
GitHub	Version control and delivery	Stores the source code, requirements, README, and report in one reproducible project location.

The implementation intentionally avoids JavaScript. The application can be run with a Python virtual environment and a single Streamlit command.

3. System workflow

The prototype follows a simple pipeline:

STEP	INPUT	PROCESS	OUTPUT
1. Catalog	Object ID, type, altitude, position, velocity	Load controlled synthetic records	Satellite and debris objects
2. Relative state	Satellite and debris coordinates	Subtract position and velocity vectors	Relative position and relative velocity
3. Encounter	Relative state	Estimate the time where distance is minimized	TCA and closest distance
4. Risk triage	Distance, altitude delta, speed	Apply transparent threshold score	LOW, MEDIUM, or HIGH
5. Presentation	Calculated encounters	Render metrics, alerts, table, and map	Faculty-ready dashboard

The top-ranked debris object is surrounded by an illustrative synthetic proximity zone on the map. This is an orbital hazard visualization, not a geographic ground-impact prediction.

Recommended demonstration order

- Start with the four headline metrics.
- Point to the high-priority alert and explain closest approach and TCA.
- Use the table to show that every result is traceable to an object.
- Show the README and this report to explain scope and limitations.

4. How object tracking works

Tracking in this MVP means maintaining a structured record for every object, recalculating its relative state against a reference satellite, and ordering the results by the urgency of the estimated encounter. The system does not need a database for this demonstration because the synthetic catalog is loaded from Python records whenever the app starts.

Tracked fields

Field	Purpose
Object ID	Stable identifier such as OG-DEB-17 so the same object can be recognized in the catalog.
Object type	Separates satellites from debris and controls how the object is displayed and scored.
Altitude	Provides an orbital-layer comparison and contributes to the demonstration risk score.
Relative position	Synthetic x and y coordinates used to place an object on the orbital map.
Velocity	Synthetic vx and vy values used to estimate how the object moves relative to the satellite.
Encounter result	Calculated distance, time to closest approach, relative speed, risk label, and risk score.

Tracking cycle

- Load the catalog at application start.
- Select the first satellite as the reference object.
- Compare each debris object's position and velocity with that satellite.
- Calculate a new encounter result for each debris record.
- Sort events from highest to lowest risk score.
- Refresh the metrics, alerts, map, and table from the sorted results.

In a real tracking system, the same cycle would be repeated whenever a new observation, TLE, or ephemeris update arrives. The new state would be propagated to a common time, uncertainty would be carried through the calculation, and the result would be reviewed by an operator.

5. Calculation method

For each debris object, OrbitGuard uses a simplified relative-motion approximation. If r_0 is the relative position and v is the relative velocity, the future relative position is represented as:

$$r(t) = r_0 + v * t$$

The estimated time of closest approach is found by minimizing the squared distance. In the implementation, the result is constrained to a 72-hour demonstration horizon. The risk score is not a certified collision probability; it is a transparent prioritization score based on:

- minimum estimated distance
- difference in orbital altitude
- relative speed

This makes the reasoning inspectable. A production system would replace the simplified approximation with high-fidelity orbit propagation and uncertainty-aware probability-of-collision methods.

Risk interpretation

Label	Meaning in this MVP
HIGH	Closest and/or fastest encounter in the synthetic horizon; requires first review.
MEDIUM	Worth monitoring, but lower priority than HIGH events.
LOW	Included in the catalog but below the demonstration priority threshold.
MONITORED	Satellite object shown in the catalog without a debris encounter score.

6. How to run and explain the demo

From the OrbitGuard repository:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

The browser will open the dashboard. No database, external API key, or JavaScript build process is required.

Likely faculty questions

Question	Short answer
Why synthetic data?	To demonstrate the complete workflow safely and reproducibly without claiming access to operational spacecraft telemetry.
Is this a real collision probability?	No. It is a transparent prioritization score. Real probability requires uncertainty covariance and validated astrodynamics.
Why Streamlit?	It lets a Python-only project provide an interactive dashboard quickly, with no JavaScript frontend.
What is the next step?	Integrate validated TLE or ephemeris data, an orbit propagator, covariance handling, and expert review.
Can it command a satellite maneuver?	No. This MVP only visualizes and prioritizes synthetic close approaches.

Honest project note

The prototype was developed with AI assistance. The student should be able to explain the code, the assumptions, and the limitations in their own words before submitting or presenting it.

Repository

<https://github.com/Shibukedang27/OrbitGuard>